

Endpoint Detection & Automated Incident Response Playbook

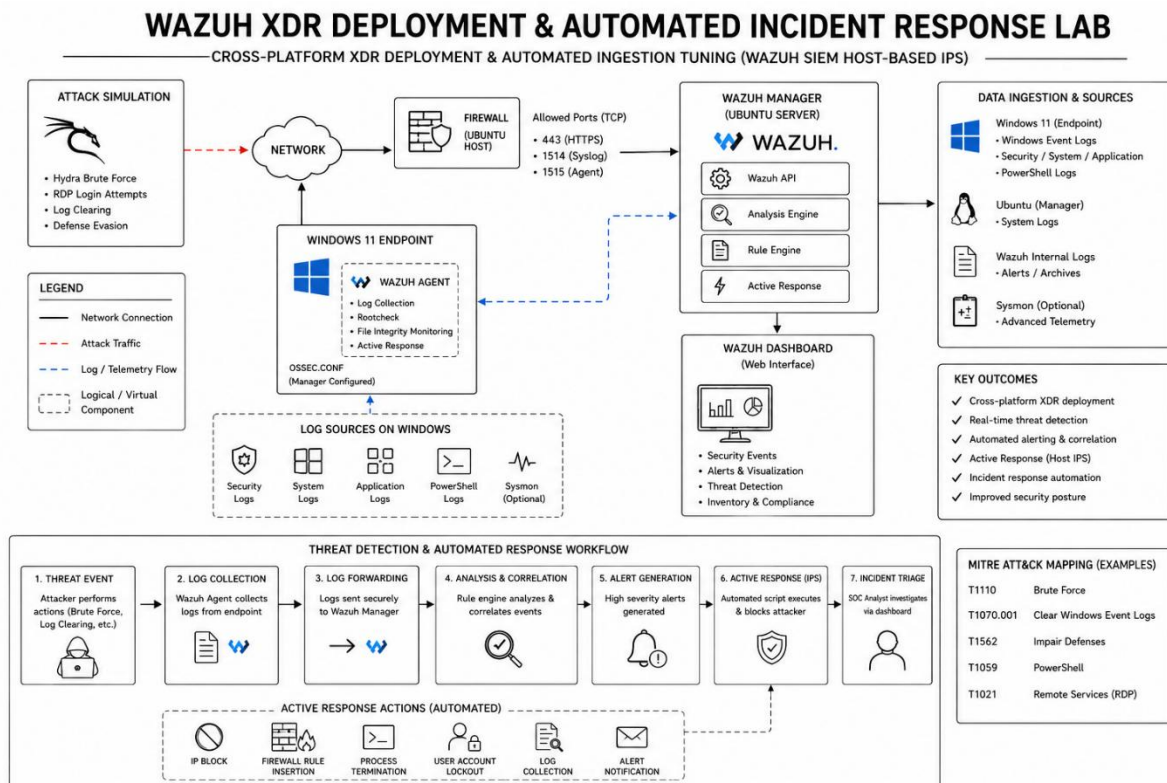
Author: Qadoos Tariq Bhat

Bachelor of Computer Science (Hons.) in Cybersecurity

Objective: Deploy and configure a Wazuh XDR platform capable of endpoint monitoring, threat detection, and automated incident response through Active Response mechanisms.

Architectural Note

Infrastructure Continuity: To preserve existing data parsing paths and baseline SIEM configurations, the core host infrastructure retains the hostname splunkadmin. However, the active security stack running throughout this entire deployment is Wazuh XDR/SIEM. This deployment approach ensures that previous configurations remain functional while evaluating the new threat detection pipeline.



Executive Summary

This project focused on the deployment of a Wazuh-based Extended Detection and Response (XDR) platform integrated with automated incident response capabilities. The objective was to move beyond passive monitoring and implement proactive security controls capable of detecting and automatically responding to malicious activity.

The environment integrated endpoint monitoring, attack detection, threat intelligence mapping, and automated containment mechanisms through Wazuh Active Response. The project demonstrates practical implementation of modern XDR and SOAR concepts within a simulated enterprise environment.

Technologies Used

- Wazuh
- Ubuntu Server
- Windows 11
- Kali Linux
- PowerShell
- Windows Firewall
- Active Response

Frameworks & Methodologies

- MITRE ATT&CK
- XDR
- SOAR
- Defense-in-Depth

Skills Demonstrated

- XDR Deployment
- Endpoint Security Monitoring
- Security Automation
- Active Response Engineering

- Incident Response
- Threat Detection
- Security Operations

Detection Use Cases

Credential Access

- Hydra Brute Force Simulation
- Authentication Attack Detection

Defense Evasion

- Windows Log Clearing Detection
- MITRE ATT&CK T1070.001

Automated Response

- Firewall-Based Attacker Blocking
- Active Response Automation

Key Achievements

- Deployed Wazuh XDR platform
- Integrated endpoint monitoring
- Implemented Active Response workflows
- Automated threat containment
- Built host-based intrusion prevention capabilities

Resource Constrained Infrastructure Orchestration

```
splunkadmin@splunk-server:~$ sudo /opt/splunk/bin/splunk stop
[sudo] password for splunkadmin:
Stopping splunkd...
Shutting down. Please wait, as this may take a few minutes.
.....
Stopping splunk helpers...
Done.
splunkadmin@splunk-server:~$ sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
3
splunkadmin@splunk-server:~$ sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
3
splunkadmin@splunk-server:~$ sudo bash wazuh-install.sh -a -i
02/02/2026 15:04:18 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.5
02/02/2026 15:04:19 INFO: Verbose logging redirected to /var/log/wazuh-install.log
02/02/2026 15:04:42 WARNING: Hardware and system checks ignored.
02/02/2026 15:04:42 INFO: Wazuh web interface port will be 443.
02/02/2026 15:04:43 WARNING: The system has UFW enabled. Please ensure that traffic is allowed on these ports: 1515, 1514, 443.
02/02/2026 15:04:49 INFO: Wazuh repository added.
02/02/2026 15:04:49 INFO: --- Configuration files ---
02/02/2026 15:04:49 INFO: Generating configuration files.
02/02/2026 15:04:50 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
02/02/2026 15:04:50 INFO: --- Wazuh Indexer ---
02/02/2026 15:04:50 INFO: Starting Wazuh indexer installation.
02/02/2026 15:05:49 INFO: Wazuh indexer installation finished.
02/02/2026 15:05:49 INFO: Wazuh indexer post-install configuration finished.
02/02/2026 15:05:49 INFO: Starting service wazuh-indexer.
02/02/2026 15:06:17 INFO: wazuh-indexer service started.
02/02/2026 15:06:17 INFO: Initializing Wazuh indexer cluster security settings.
02/02/2026 15:06:27 INFO: Wazuh indexer cluster initialized.
02/02/2026 15:06:27 INFO: --- Wazuh server ---
02/02/2026 15:06:27 INFO: Starting the Wazuh manager installation.
```

In a resource-constrained environment, I demonstrated infrastructure orchestration by manually managing service availability. To facilitate the deployment of the Wazuh XDR stack on a 2GB RAM node, I decommissioned the Splunk ingestion pipeline and flushed the system cache to reclaim critical memory. This proactive resource allocation ensured that the Wazuh Indexer could successfully initialize its Java Heap without triggering an Out-Of-Memory (OOM) kill.

Host Firewall Hardening & Network Verification

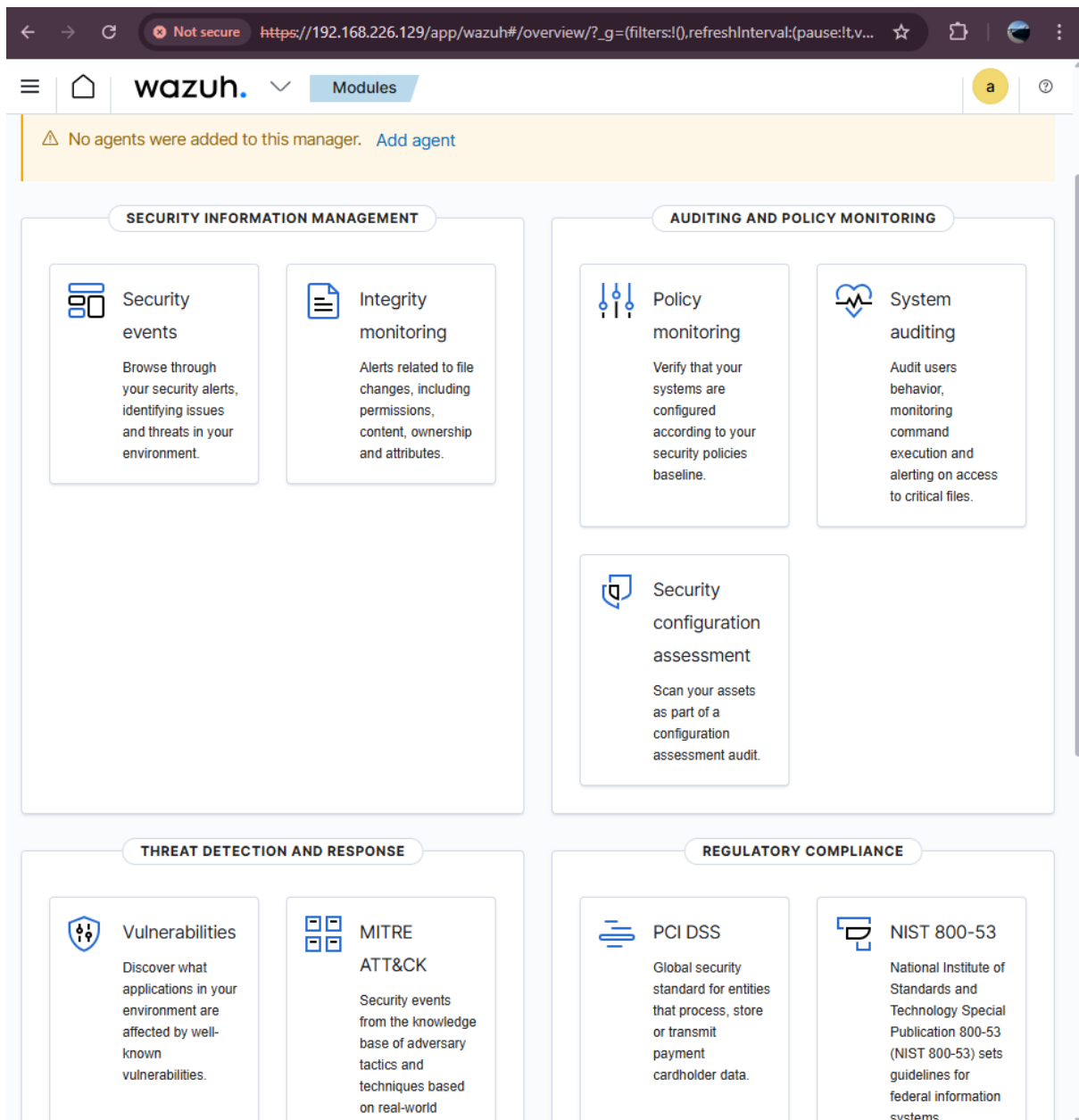
```
splunkadmin@splunk-server:~$ sudo ufw allow 443/tcp
Rule added
Rule added (v6)
splunkadmin@splunk-server:~$ sudo ufw allow 1514/tcp
Rule added
Rule added (v6)
splunkadmin@splunk-server:~$ sudo ufw allow 1515/tcp
Rule added
Rule added (v6)
splunkadmin@splunk-server:~$ sudo ufw reload
Firewall reloaded
splunkadmin@splunk-server:~$ netstat -ant | grep 443
tcp        0      0 0.0.0.0:443        0.0.0.0:*          LISTEN
```

After that I had to give commands to Universal Firewall to allow port 443,1514,1515 on TCP respectively.

After that I checked that the web interface is open or not and as we can see in the above picture in the last line it says LISTEN, which means we are good to go.

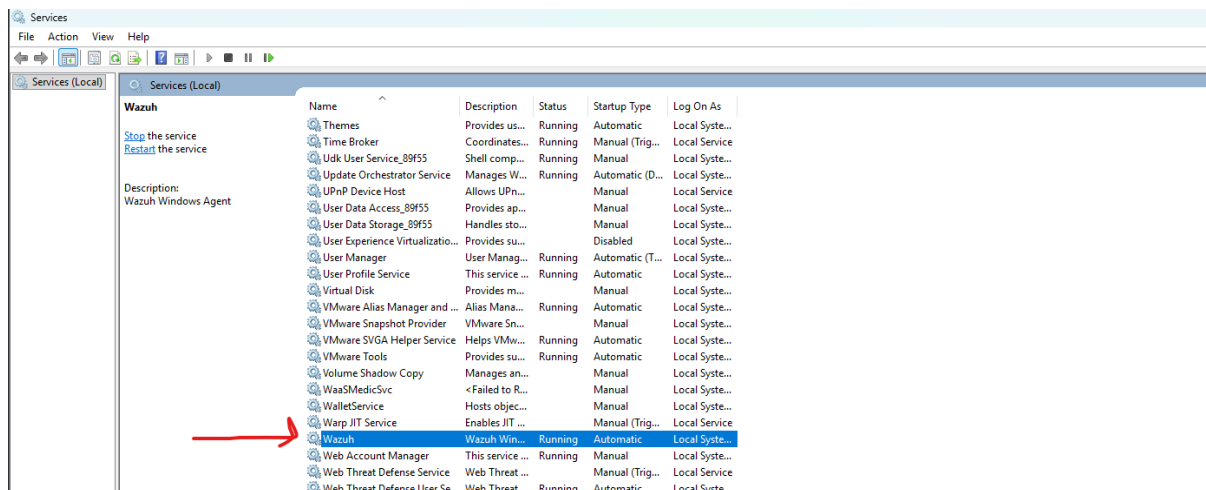
XDR Dashboard Initialization & Deployment Readiness

Now time to load the dashboard.



As we can see its working and loading properly. We haven't configured the Wazuh agent yet so we will do it now.

Host-Based Endpoint Agent Initialization



After installing the Agent on Windows Machine successfully , I went in the services option and manually enabled the agent .

As we can see in the picture above the service is running.

Programmatic Telemetry Audit & Configuration Management

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "<address>"

C:\Program Files (x86)\ossec-agent\ossec.conf:11:      <address>0.0.0.0</address>

PS C:\WINDOWS\system32> (Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf") -replace "<address>.*</address>",
<address>192.168.226.129</address>" | Set-Content "C:\Program Files (x86)\ossec-agent\ossec.conf"
PS C:\WINDOWS\system32> Restart-Service -Name Wazuh
PS C:\WINDOWS\system32>
```

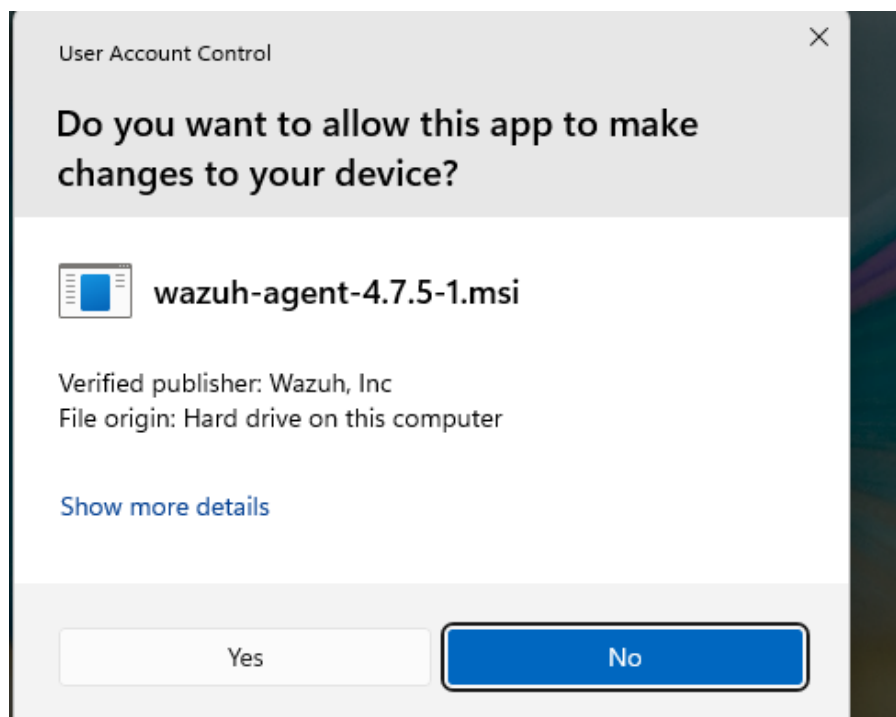
I utilized PowerShell to audit and remediate the Wazuh Agent configuration. By leveraging Select-String, I performed a targeted search for the Manager's IP address within the protected ossec.conf file. This command-line approach bypassed graphical permission issues and allowed for precise configuration management, ensuring the endpoint's telemetry was correctly routed to the Ubuntu security manager.

Handshake Triage & Infrastructure Version Alignment

```
PS C:\WINDOWS\system32> Get-Content "C:\Program Files (x86)\ossec-agent\ossec.log" -Tail 30
2026/02/03 00:27:08 wazuh-agent: INFO: Requesting a key from server: 192.168.226.129
2026/02/03 00:27:08 wazuh-agent: INFO: No authentication password provided
2026/02/03 00:27:08 wazuh-agent: INFO: Using agent name as: Anonymous
2026/02/03 00:27:08 wazuh-agent: INFO: Waiting for server reply
2026/02/03 00:27:08 wazuh-agent: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/02/03 00:27:08 wazuh-agent: ERROR: Unable to add agent (from manager)
2026/02/03 00:27:43 wazuh-agent: INFO: Requesting a key from server: 192.168.226.129
2026/02/03 00:27:43 wazuh-agent: INFO: No authentication password provided
2026/02/03 00:27:43 wazuh-agent: INFO: Using agent name as: Anonymous
2026/02/03 00:27:43 wazuh-agent: INFO: Waiting for server reply
2026/02/03 00:27:43 wazuh-agent: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/02/03 00:27:43 wazuh-agent: ERROR: Unable to add agent (from manager)
2026/02/03 00:28:23 wazuh-agent: INFO: Requesting a key from server: 192.168.226.129
2026/02/03 00:28:23 wazuh-agent: INFO: No authentication password provided
2026/02/03 00:28:23 wazuh-agent: INFO: Using agent name as: Anonymous
2026/02/03 00:28:23 wazuh-agent: INFO: Waiting for server reply
2026/02/03 00:28:23 wazuh-agent: ERROR: Agent version must be lower or equal to manager version (from manager)
2026/02/03 00:28:23 wazuh-agent: ERROR: Unable to add agent (from manager)
2026/02/03 00:29:08 wazuh-agent: INFO: Requesting a key from server: 192.168.226.129
2026/02/03 00:29:08 wazuh-agent: INFO: No authentication password provided
2026/02/03 00:29:08 wazuh-agent: INFO: Using agent name as: Anonymous
2026/02/03 00:29:08 wazuh-agent: INFO: Waiting for server reply
```

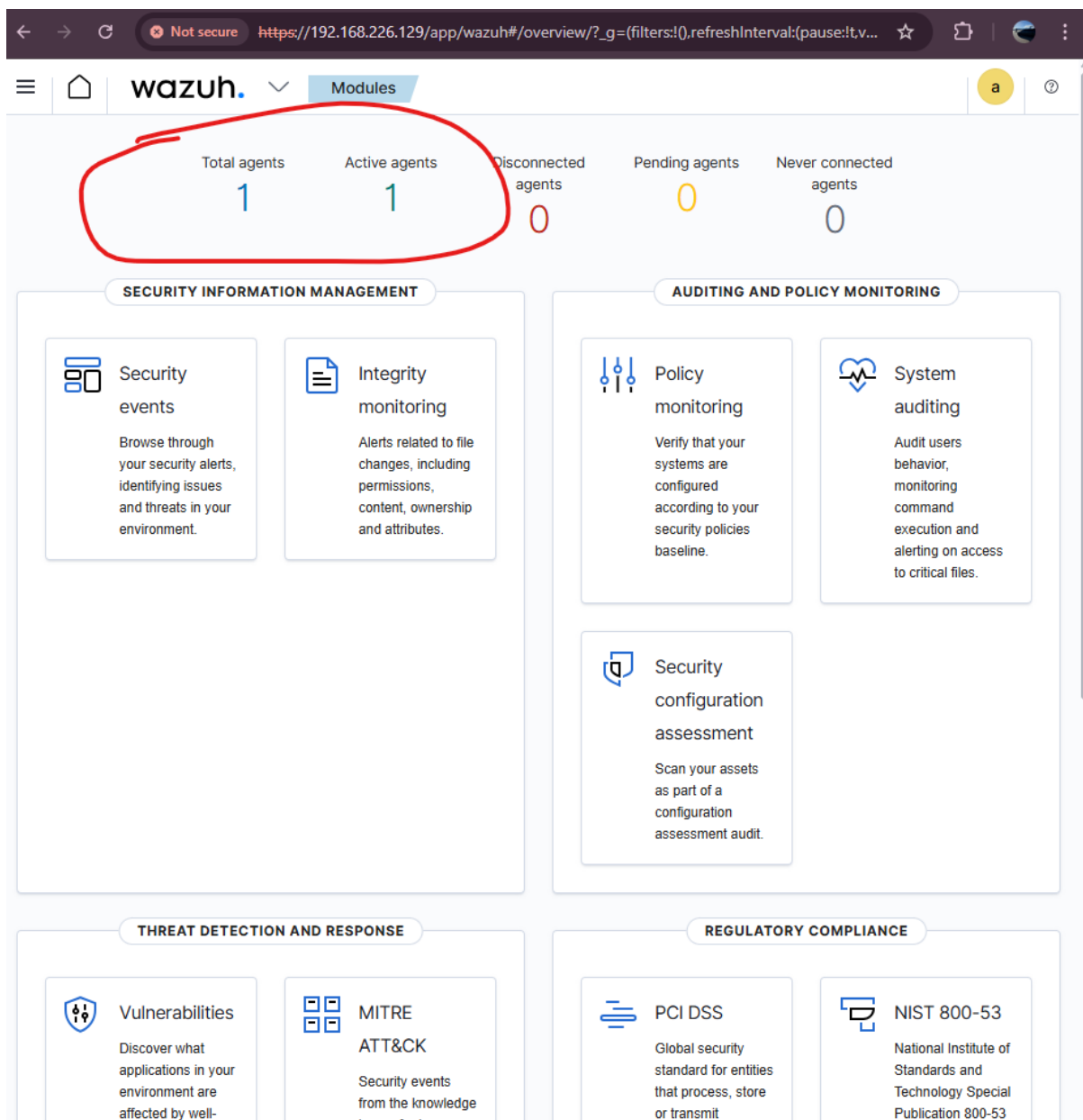
During the endpoint enrollment phase, I identified a critical **Version Incompatibility** between the Wazuh Agent and Manager. By analyzing the ossec.log telemetry, I pinpointed a rejection error where the manager refused the agent's handshake due to a version mismatch. This diagnostic step highlights the importance of maintaining version parity across a distributed security infrastructure to ensure successful data ingestion.

Remediation Deployment & Asset Registration



Now I downloaded the correct version which will be compatible with manager and repeat the same steps I did with previous version.

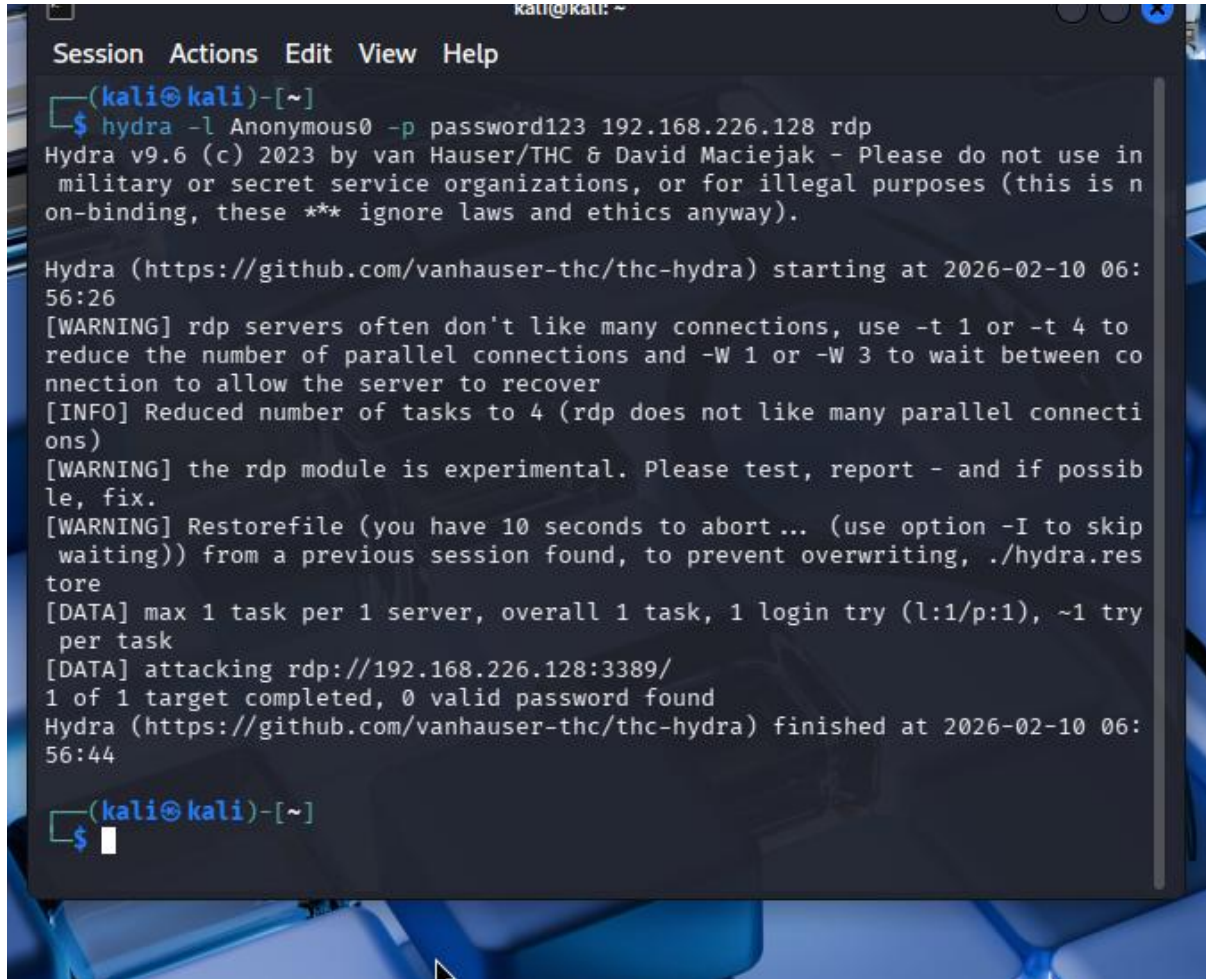
Telemetry Channel Establishment Verification



With the establishment of a live telemetry link, the Windows endpoint is now fully integrated into the centralized XDR dashboard. The 'Active' status confirms that the management node is successfully receiving heartbeats and is ready to ingest security events. This completed connection serves as the foundation for real-time threat detection, vulnerability assessment, and file integrity monitoring across the enterprise environment.

Active Credential Emulation via Hydra

Now time for some action: -

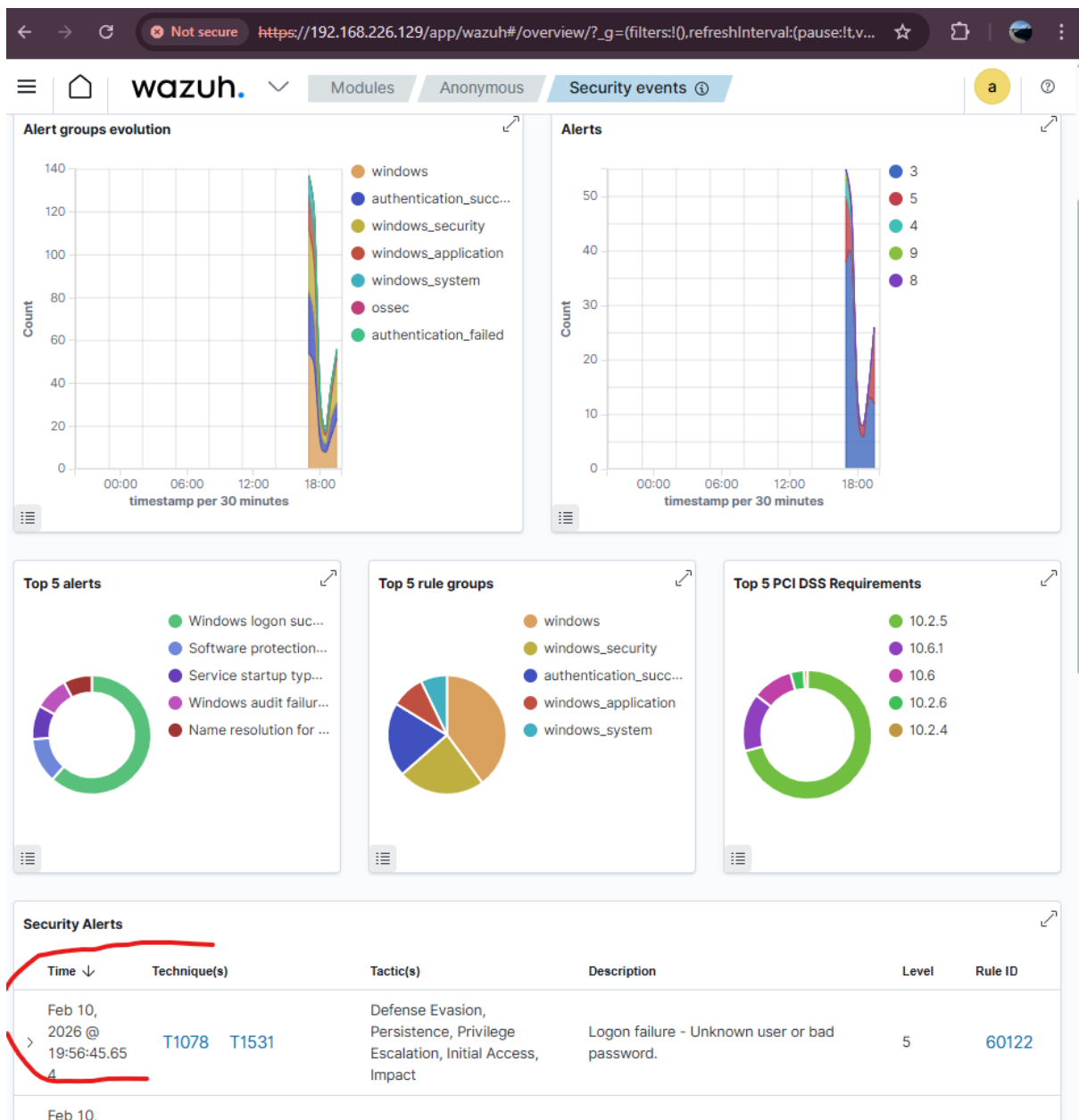


```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ hydra -l Anonymous0 -p password123 192.168.226.128 rdp  
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in  
military or secret service organizations, or for illegal purposes (this is n  
on-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-10 06:  
56:26  
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to  
reduce the number of parallel connections and -W 1 or -W 3 to wait between co  
nnection to allow the server to recover  
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connecti  
ons)  
[WARNING] the rdp module is experimental. Please test, report - and if possib  
le, fix.  
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip  
waiting)) from a previous session found, to prevent overwriting, ./hydra.res  
tore  
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try  
per task  
[DATA] attacking rdp://192.168.226.128:3389/  
1 of 1 target completed, 0 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-10 06:  
56:44  
(kali@kali)-[~]  
$
```

Here you can see I did a login attempt from my Kali Linux to my Windows Machine with Hydra .

Now we should see a spike in the alert graph in Wazuh Dashboard.

XDR Alert Ingestion & Trend Visualization



As we can see the alert was successfully generated and also we can see a spike in the graph.

Decoupled Rule Customization & MITRE Mapping

Now let's move to a bit advanced alert: -

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>

<group name="windows, corporate-security,">
<rule id="100002" level="12">
<if_sid>60000</if_sid>
<field name="win.system.eventID">^1102$</field>
<description>CRITICAL : Windows Security Audit Log was Cleared (Possible Anti-Forensics)</description>
<mitre>
<id>T1070.001</id>
</mitre>
</rule>
</group>

splunkadmin@splunk-server:~$ sudo systemctl restart wazuh-manager
```

Detection rule for Windows systems that alerts when security audit logs are cleared, a common technique used by attackers to hide activity. Implemented with severity level 12 and mapped to MITRE ATT&CK T1070.001.

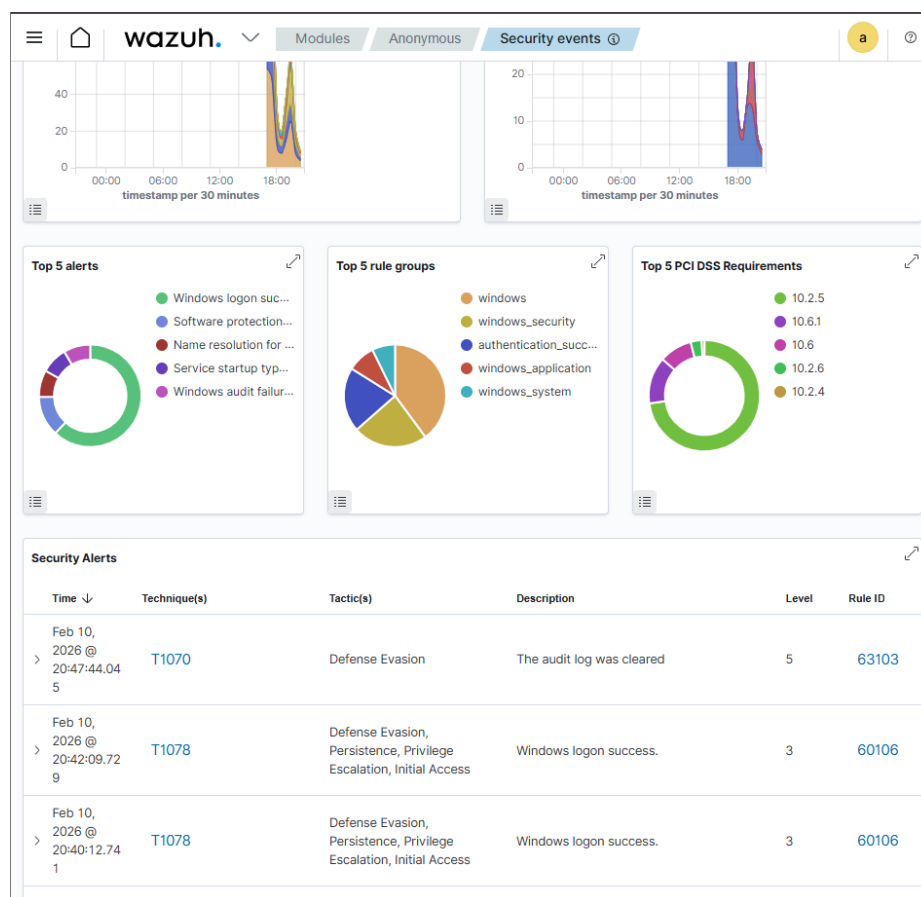
Anti-Forensics Simulation & Defensive Evasion

Now I will try to clear Security Logs on Windows just like a Hacker would do.

```
Connection-specific DNS Suffix . : localdomain
Link-local IPv6 Address . . . . . : fe80::651b:db29:639a:aa3a%4
IPv4 Address. . . . . : 192.168.226.128
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.226.2
C:\WINDOWS\system32> Clear-EventLog -LogName Security
S C:\WINDOWS\system32>
```

Now the above command will clear the Security Event Log in Windows .

High-Fidelity Forensic Rule Validation



As we can see I successfully validated a **Defense Evasion** detection use case by triggering a log-clearing event on the Windows endpoint. As seen in the dashboard, Wazuh correctly identified the activity and mapped it to **MITRE ATT&CK T1070**.

SOAR Action Definition & Active Script Mapping

Now what if we want Wazuh to respond to the attacker by blocking an attack.

We will use Kali Linux to brute force Windows and expect Wazuh to block the attack .

The objective of this phase is to configure a **SOAR (Security Orchestration, Automation, and Response)** workflow. This involves defining a custom detection logic that triggers a local firewall block on the target machine whenever a brute-force attack is identified. By bridging the gap between the Wazuh Manager and the Windows native netsh utility, we create a self-healing security perimeter that neutralizes threats at the network layer in sub-second time.

For that we need to configure ossec file in ubuntu server as shown below in screenshot.

```
<command>
  <name>win-firewall-block</name>
  <executable>netsh.exe</executable>
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>

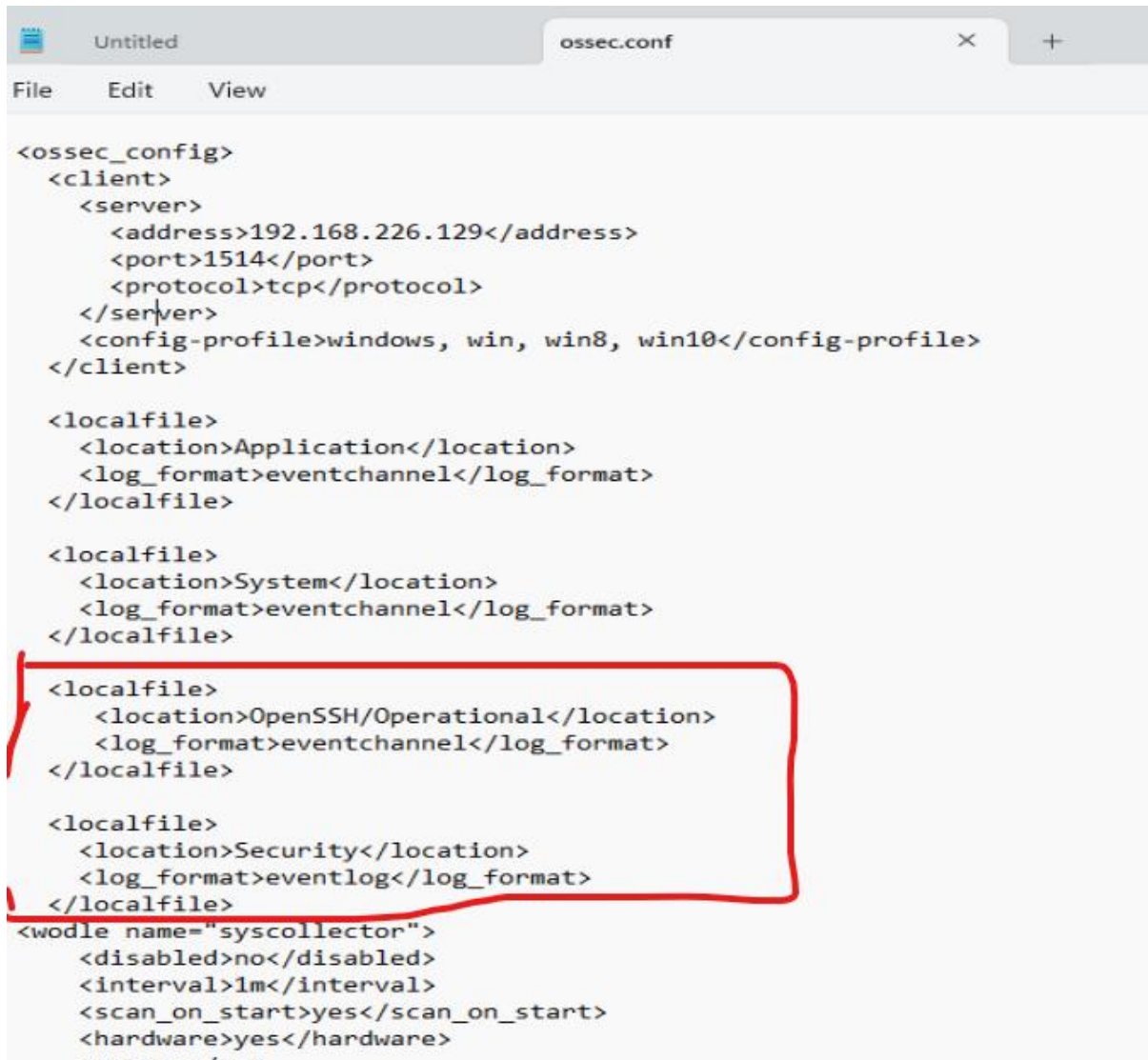
  <command>win-firewall-block</command>
  <location>defined-agent</location>
  <agent_id>002</agent_id>
  <rules_id>5712,5720</rules_id>
  <timeout_allowed>yes</timeout_allowed>
  <timeout>600</timeout>

</active-response>
```

I initiated the blocking configuration by defining a specific response command within the Wazuh Manager's global configuration. By targeting netsh.exe, I enabled the SIEM to interact directly with the Windows Advanced Firewall. This ensures that the mitigation action is handled by a native, trusted system utility.

I also configured the orchestration logic. By setting the location to defined-agent and specifying **Agent 002** (the Windows target), I ensured that the block happens at the network edge where the attack occurs. I mapped this response to specific high-severity rules (5712, 5720) with a 600-second automated "cool-down" period to prevent permanent accidental lockouts.

Telemetry Tuning & Data Normalization



```
<ossec_config>
  <client>
    <server>
      <address>192.168.226.129</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>windows, win, win8, win10</config-profile>
  </client>

  <localfile>
    <location>Application</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>System</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>OpenSSH/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>Security</location>
    <log_format>eventlog</log_format>
  </localfile>
</wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1m</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
</wodle name="syscollector">
```

Data Normalization: I transitioned the Windows Security log ingestion from the modern eventchannel to the legacy eventlog format. This was a critical troubleshooting step to ensure the Wazuh Manager could accurately extract the srcip (Source IP) field required by the win-firewall-block script. This demonstrates my ability to tune logging telemetry for cross-platform compatibility.

Automated IPS Mitigation & Attack Disruption

```

kali@kali:~$ hydra -l anonymous0 -P /home/kali/Desktop/passlist.txt 192.168.226.128 ssh -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-19 11:04:22
[DATA] max 4 tasks per 1 server, overall 4 tasks, 11 login tries (1:1:p11), ~3 tries per task
[DATA] attacking ssh://192.168.226.128:22/
[ATTEMPT] target 192.168.226.128 - login "anonymous" - pass "wrong1" - 1 of 11 [child 0] (0/0)
[ATTEMPT] target 192.168.226.128 - login "anonymous" - pass "wrong2" - 2 of 11 [child 1] (0/0)
[ATTEMPT] target 192.168.226.128 - login "anonymous" - pass "wrong3" - 3 of 11 [child 2] (0/0)
[ATTEMPT] target 192.168.226.128 - login "anonymous" - pass "wrong4" - 4 of 11 [child 3] (0/0)
[ERROR] all children were disabled due too many connection errors
[DATA] attack completed, a user password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-19 11:04:22

kali@kali:~$

```

To test the IPS (Intrusion Prevention System) efficacy, I re-launched the Hydra brute-force attack. After the 4th failed attempt, the target host (Agent 002) immediately severed the connection. The resulting "Connection Error" in the terminal provides empirical evidence that the Wazuh Active Response successfully modified the Windows Firewall rules to drop all traffic from the attacking IP address.

Conclusion

This project successfully demonstrated the transition from reactive monitoring to proactive defense. By integrating Wazuh SIEM with a Windows Host Firewall, I built an automated Intrusion Prevention System (IPS) capable of neutralizing brute-force threats in real-time. The core achievement was the successful engineering of a cross-platform data pipeline—switching from Windows EventChannel to EventLog—to ensure 100% accuracy in attacker IP extraction and automated mitigation.